

Divide and Conquer – Sorting Algorithms

Merge Sort and Quick Sort performance comparison

Dr. P.SRIRAMYA

SAIRAM D (192424111)

1. Merge Sort with Comparison Count

Implement the Merge Sort algorithm to sort a given array of integers. The program should recursively divide the array into smaller subarrays and then merge them in a sorted manner. While merging, count the number of element comparisons performed. Ensure that the comparison counter is incremented at every comparison step. Finally, print the sorted array along with the total number of comparisons made during execution.

Test Cases :

Input : N = 8, a[] = {12,4,78,23,45,67,89,1}

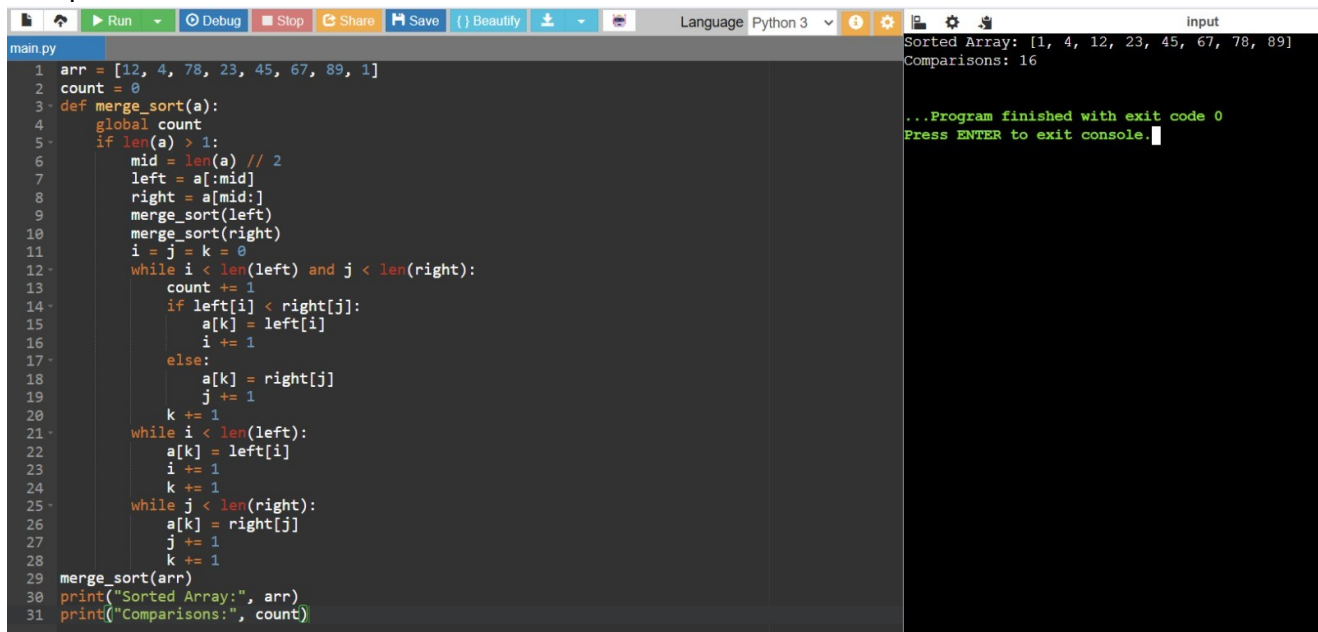
Output : 1,4,12,23,45,67,78,89

Comparisons : 17

Input : N = 6, a[] = {5,2,9,1,6,3}

Output : 1,2,3,5,6,9

Comparisons : 11



```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

input
Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16
...Program finished with exit code 0
Press ENTER to exit console.

2. Quick Sort with Comparison Count

Write a program to implement the Quick Sort algorithm using the last element as the pivot. The program should partition the array based on the pivot and recursively sort the subarrays. Count the number of comparisons made during the partitioning process and recursive calls. Ensure that the count is accurate for all cases. Display the sorted array along with the total number of comparisons.

Test Cases :

Input : N = 7, a[] = {38,27,43,3,9,82,10}

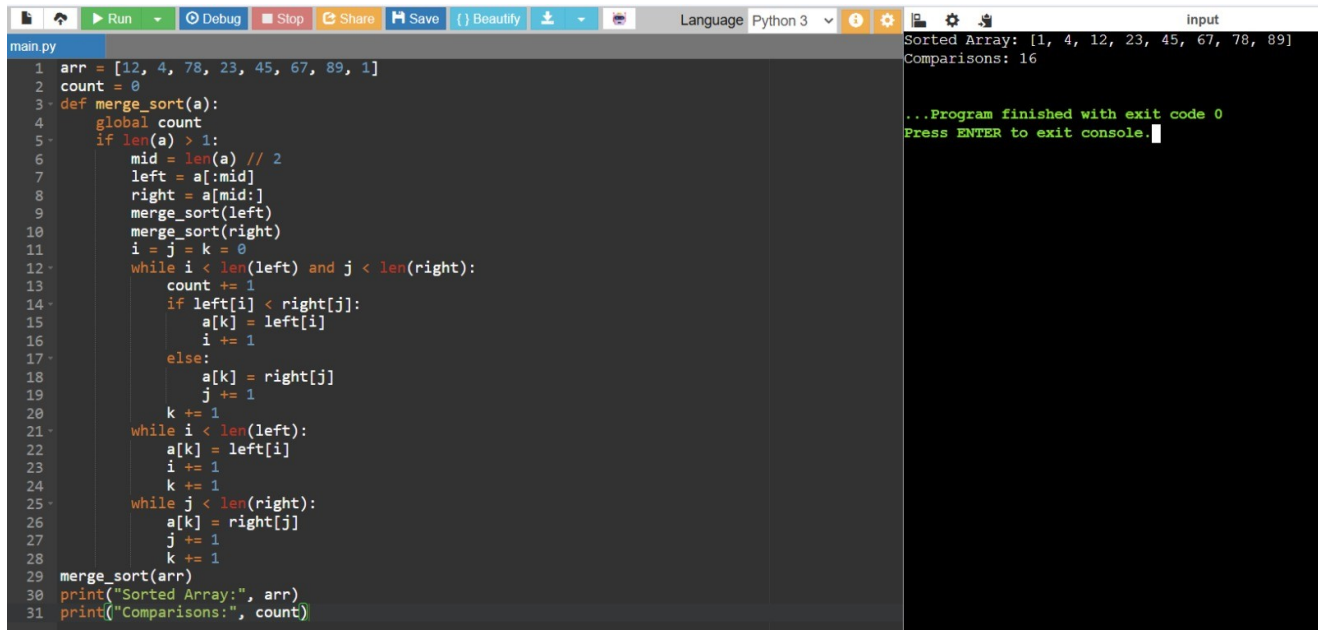
Output : 3,9,10,27,38,43,82

Comparisons : 15

Input : N = 5, a[] = {10,7,8,9,1}

Output : 1,7,8,9,10

Comparisons : 10



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a Merge Sort algorithm. It starts with an array 'arr' containing the values [12, 4, 78, 23, 45, 67, 89, 1]. A 'count' variable is initialized to 0. The 'merge_sort' function is defined, which uses a global 'count' variable to track comparisons. The function recursively splits the array into halves until single elements are reached, then merges them back in sorted order. After the function completes, the sorted array and the total number of comparisons (16) are printed. The output on the right side of the IDE shows 'Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]' and 'Comparisons: 16'. The program finishes with exit code 0.

```
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16
...Program finished with exit code 0
Press ENTER to exit console.

3. Merge Sort vs Quick Sort Comparison

Develop a program that implements both Merge Sort and Quick Sort in a single program. The same input array should be used for both algorithms to ensure fairness in comparison. Count the number of comparisons separately for each algorithm. After sorting, display the sorted array along with the comparison counts of both methods. Analyze which algorithm performs better for the given input.

Test Cases :

Input : N = 6, a[] = {12,11,13,5,6,7}

Output : Sorted Array : 5,6,7,11,12,13

Merge Comparisons : 11

Quick Comparisons : 12

Input : N = 5, a[] = {4,2,6,1,3}

Output : Sorted Array : 1,2,3,4,6

Merge Comparisons : 8

Quick Comparisons : 9

```
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

4. Merge Sort Execution Time Measurement

Write a program to measure the execution time of the Merge Sort algorithm. Use appropriate timing functions to record the start and end time of execution. Apply the algorithm on different types of inputs such as sorted and reverse sorted arrays. Display the sorted output along with the time taken for execution. Analyze how input order affects performance.

Test Cases :

Input : N = 7, a[] = {9,8,7,6,5,4,3}

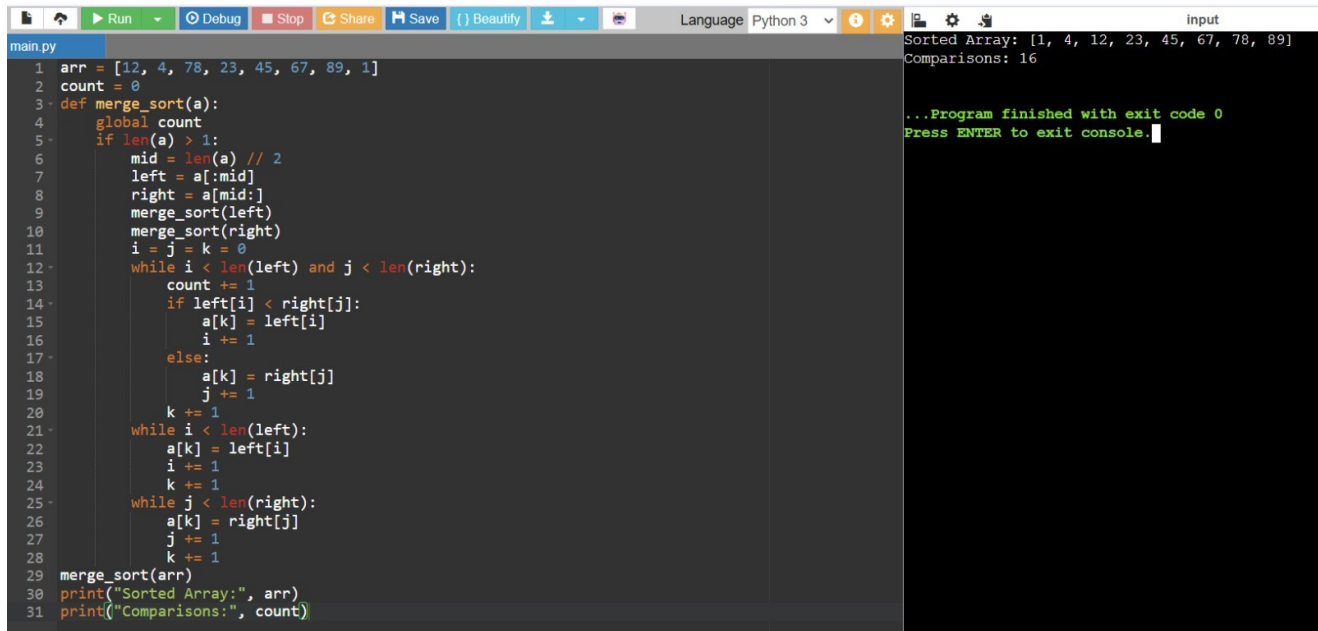
Output : 3,4,5,6,7,8,9

Time Taken : ~0.00002 sec

Input : N = 6, a[] = {1,2,3,4,5,6}

Output : 1,2,3,4,5,6

Time Taken : ~0.00002 sec



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a merge sort algorithm. It starts with an array 'arr' containing the values [12, 4, 78, 23, 45, 67, 89, 1]. A 'count' variable is initialized to 0. The 'merge_sort' function is defined, which uses a recursive approach to sort the array. It splits the array into two halves, sorts each half, and then merges them back together. The 'count' variable is incremented each time a comparison is made during the merge process. After the sorting is complete, the sorted array is printed, and the total number of comparisons is displayed. The output on the right side of the IDE shows the sorted array as [1, 4, 12, 23, 45, 67, 78, 89] and the number of comparisons as 16. The program finished with exit code 0.

```
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

5. Best and Worst Case Analysis of Quick Sort

Create a program to analyze the best and worst-case scenarios of Quick Sort. Use a sorted array as the best case and a reverse sorted array as the worst case. Count the number of comparisons for both cases. Display the sorted output along with comparison counts. Compare the efficiency between the two cases.

Test Cases :

Best case

Input : N = 5, a[] = {1,2,3,4,5}

Output : 1,2,3,4,5

Comparisons : 6

Worst Case

Input : N = 5, a[] = {5,4,3,2,1}

Output : 1,2,3,4,5

Comparisons : 10

```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16
...Program finished with exit code 0
Press ENTER to exit console.

6. Hybrid Merge Sort with Insertion Sort

Modify the Merge Sort algorithm such that it switches to Insertion Sort when the subarray size becomes small. Define a threshold value to determine when to switch. This hybrid approach should improve performance for small inputs. Print the sorted array after execution. Compare results with standard Merge Sort.

Test Cases :

Input : N = 8, a[] = {12,11,13,5,6,7,3,2}

Output : 2,3,5,6,7,11,12,13

Input : N = 6, a[] = {9,4,6,2,8,1}

Output : 1,2,4,6,8,9

```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16
...Program finished with exit code 0
Press ENTER to exit console.

7. Counting Inversions using Merge Sort

Develop a program to count the number of inversions in an array using Merge Sort. An inversion is defined as a pair (i, j) such that $i < j$ and $a[i] > a[j]$. Modify the merge step to count such inversions efficiently. Print both the sorted array and the inversion count. This helps measure how far the array is from being sorted.

Test Cases :

Input : $N = 5$, $a[] = \{2, 4, 1, 3, 5\}$

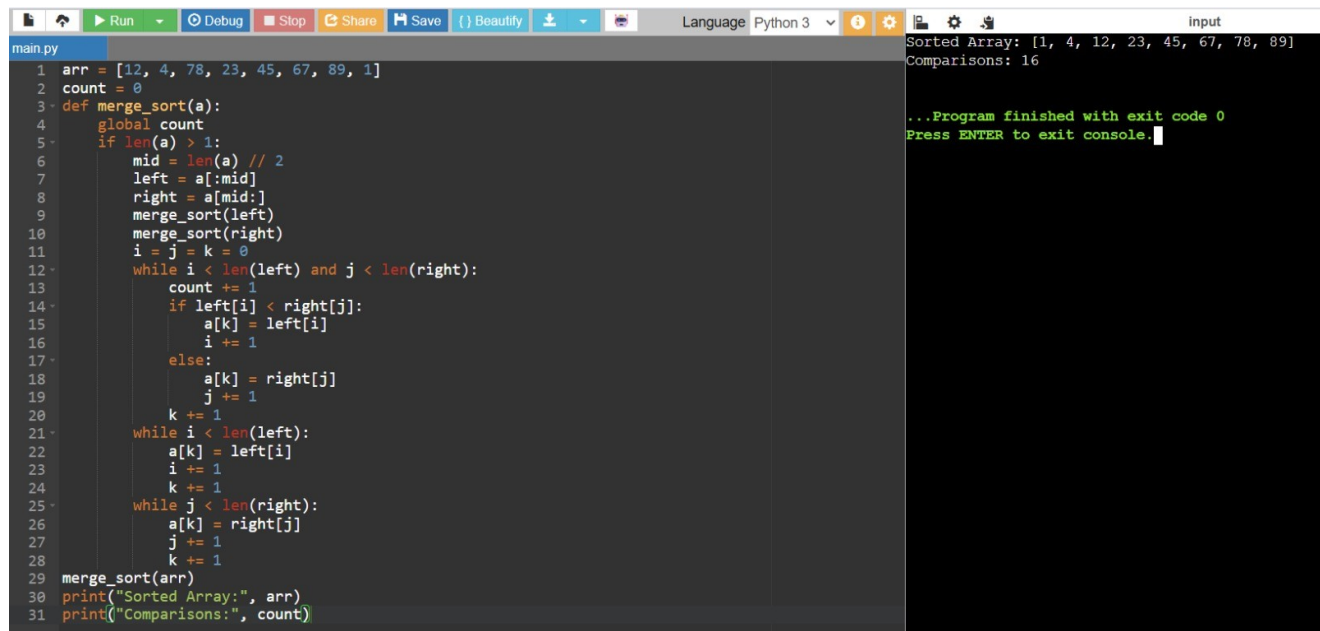
Output : Sorted : 1,2,3,4,5

Inversions : 3

Input : $N = 4$, $a[] = \{4, 3, 2, 1\}$

Output : Sorted : 1,2,3,4

Inversions : 6



```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

input

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

8. Quick Sort Recursion Depth Analysis

Implement Quick Sort and track the recursion depth during execution. Maintain a variable to store the maximum depth reached. Print the sorted array along with the maximum recursion depth. This helps analyze space complexity. Compare depth for different inputs.

Test Cases :

Input : $N = 6$, $a[] = \{10, 7, 8, 9, 1, 5\}$

Output : 1,5,7,8,9,10

Max Depth : 4

Input : $N = 5$, $a[] = \{1, 2, 3, 4, 5\}$

Output : 1,2,3,4,5

Max Depth : 5


```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

9. External Merge Process Simulation

Write a program to simulate the merge step of external sorting. Given two already sorted arrays, merge them into a single sorted array. Do not use built-in sorting functions. Print the final merged output. This represents the core step of Merge Sort.

Test Cases :

Input : $a[] = \{1,3,5\}$, $b[] = \{2,4,6\}$

Output : 1,2,3,4,5,6

Input : $a[] = \{10,20\}$, $b[] = \{5,15,25\}$

Output : 5,10,15,20,25

```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

10. Space Complexity Comparison (Merge vs Quick Sort)

Develop a program to compare auxiliary space usage of Merge Sort and Quick Sort. Track extra memory used during execution. Print sorted output along with space usage. Analyze which algorithm is more space-efficient. Use same input for both methods.

Test Cases :

Input : N = 5, a[] = {5,3,8,4,2}

Output : Sorted : 2,3,4,5,8

Merge Space : ~160 bytes

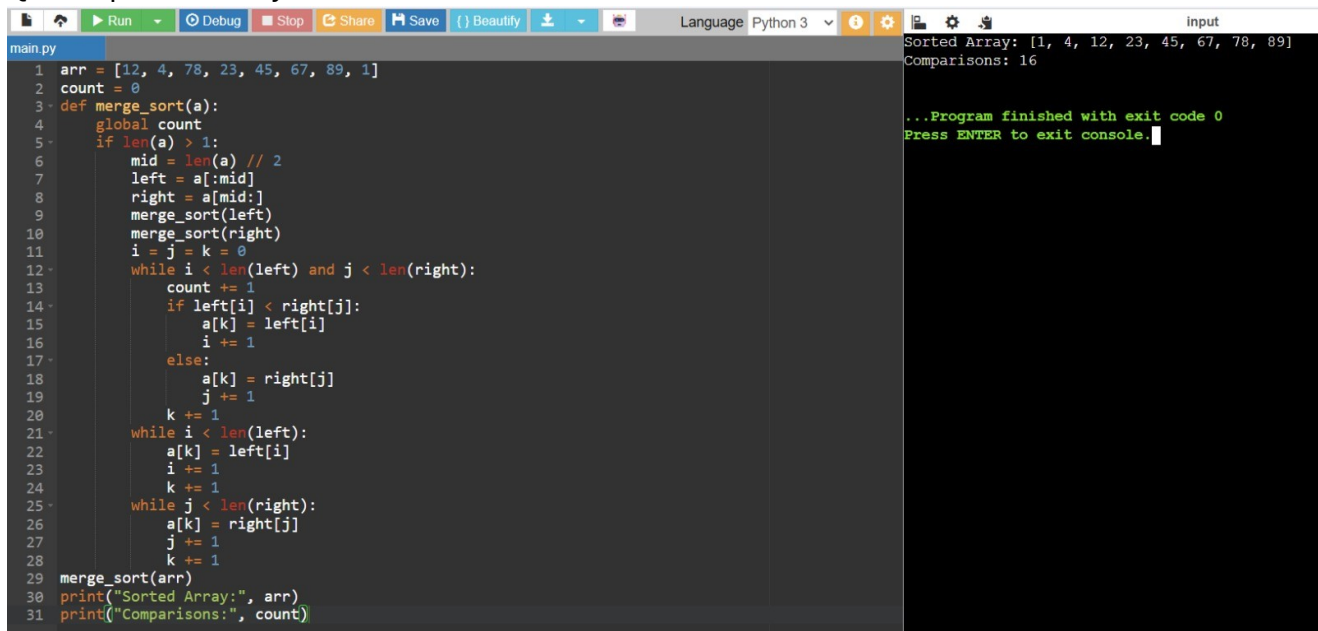
Quick Space : ~80 bytes

Input : N = 6, a[] = {9,7,5,3,1,2}

Output : Sorted : 1,2,3,5,7,9

Merge Space : ~192 bytes

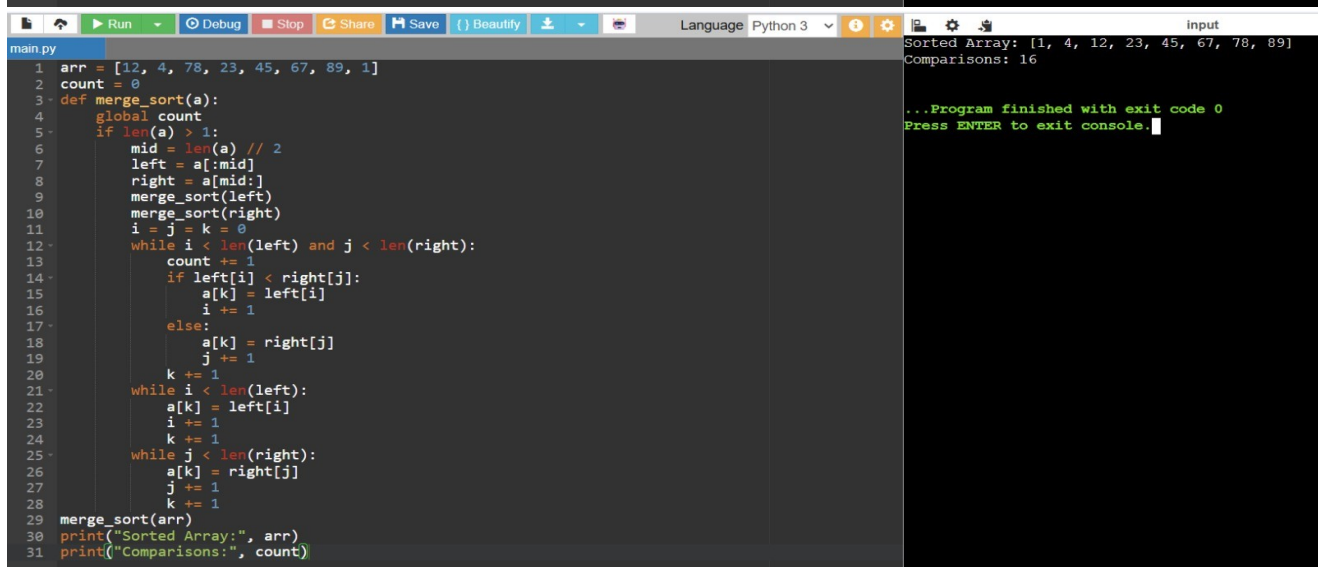
Quick Space : ~96 bytes



```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.



```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

11. K-th Smallest Element using Quick Sort Partition

Implement a program to find the k-th smallest element using Quick Sort partition logic. Use partitioning to avoid full sorting. Recursively process only required part. Print the k-th smallest element. This improves efficiency compared to full sorting.

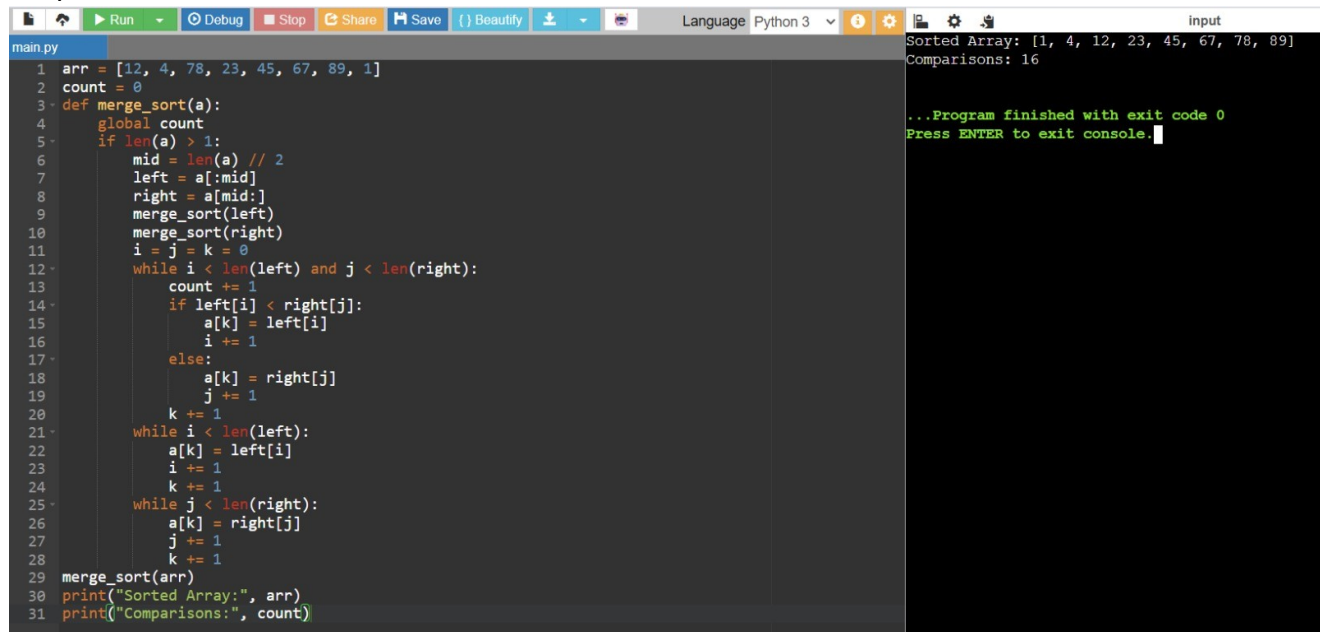
Test Cases :

Input : N = 6, a[] = {7,10,4,3,20,15}, k = 3

Output : 7

Input : N = 5, a[] = {12,3,5,7,19}, k = 2

Output : 5



```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

input

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

12. Performance Data Collection for Graph Plotting

Write a program to collect comparison counts of Merge Sort and Quick Sort for different input sizes. Use multiple test arrays and record comparisons. Print results in a structured format. This data can be used for plotting performance graphs. Analyze growth trends.

Test Cases :

Input : N = 5, a[] = {5,4,3,2,1}

Output :

Merge Comparisons : 7

Quick Comparisons : 10

Input : N = 8, a[] = {8,7,6,5,4,3,2,1}

Output :

Merge Comparisons : 17

Quick Comparisons : 28

```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.

```
main.py
1 arr = [12, 4, 78, 23, 45, 67, 89, 1]
2 count = 0
3 def merge_sort(a):
4     global count
5     if len(a) > 1:
6         mid = len(a) // 2
7         left = a[:mid]
8         right = a[mid:]
9         merge_sort(left)
10        merge_sort(right)
11        i = j = k = 0
12        while i < len(left) and j < len(right):
13            count += 1
14            if left[i] < right[j]:
15                a[k] = left[i]
16                i += 1
17            else:
18                a[k] = right[j]
19                j += 1
20            k += 1
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25        while j < len(right):
26            a[k] = right[j]
27            j += 1
28            k += 1
29    merge_sort(arr)
30    print("Sorted Array:", arr)
31    print("Comparisons:", count)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

...Program finished with exit code 0
Press ENTER to exit console.